

Resilience

Building agents by building their harness

Lesson 8 of 10 · Production

Where we left off

The agent is watched, guarded, and boxed.

Everything so far assumed things **work**.

They don't always

- the API says "overloaded"
- the network drops
- a command hangs
- the machine restarts mid-run

The cost of a failure

Ten minutes into a task, one call fails.

The run **dies**.

Everything it did is gone.

Resilience

Keep going when something fails, or pick up where you stopped.

Where does it live?

Resilience is built on two primitives:

model interface and **output**.

Why model interface

Failures happen **talking to the model**.

Lesson 1: how the harness calls the model.

Lesson 1 listed failure handling.

Why output

Failures happen **running tools**.

Lesson 2 listed failure and hang.

Report it. Don't crash.

What Lesson 1 already did

`model_interface.py` had:

- `timeout=120`
- `max_retries=3`
- a fallback model

What was missing

That was a standalone file.

The working `quark.py` still had none of it.

Today it gets all of it.

The worked example

`production/08-resilience/quark.py`

Lesson 7's `quark.py` plus resilience.

135 lines.

Three problems

1. the **model call** fails
2. a **tool** misbehaves
3. the **process** dies

Part 1: the model call

A function: ask()

All model calls go through `ask()` .

It owns the retrying and the fallback.

A list of models

```
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]
```

First choice, then backup.

The client

```
Anthropic(timeout=300, max_retries=3)
```

Wait up to 5 minutes.

The SDK retries 3 times, with backoff.

Lesson 1 vs now

Lesson 1: `timeout=120` .

Now: `timeout=300` .

A reply of 16,000 tokens can take minutes to write.

What counts as a failure

- `APIConnectionError`: couldn't reach it
- `APIStatusError` with status **429** or **500 and up**

429 and 5xx

429: you're being rate limited.

5xx: the server has a problem.

Both are worth trying again or elsewhere.

What isn't a failure to absorb

A **400**, a bad request.

Retrying won't help.

It raises, loudly.

A bug the tests caught

`OverloadedError` is the **529** status.

It is **not** a subclass of `InternalServerError` .

So we test the number

Not the class name.

```
status_code == 429 or status_code >= 500
```

Works for every case, including 529.

Falling back

First model fails after retries?

Try the next in `MODELS` .

The failure goes in the trace

Event: `model_failed` .

Lesson 5 pays off.

You can see which model failed, and when.

When everything is down

Every model fails: raise `Down` .

The run stops cleanly.

Not a stack trace in the middle.

Part 2: tools

A tool returns an error

Lesson 2: report the failure, don't crash.

Same here.

The error goes back as a `tool_result`.

Cut-off tool calls

`stop_reason` is `max_tokens` .

The last tool request may be **half written**.

Half a command is worse than none.

So it isn't run

Lesson 2's rule, now in `quark.py` .

The result says it was **cut off**, so it was not run.

Undecodable output

```
errors="replace"
```

A command prints bytes that aren't valid text.

Replaced. No crash.

Part 3: the process dies

A crash at the wrong moment

The harness is killed.

Power goes. The terminal closes. `kill -9`.

Working memory was in RAM.

Save as you go

`save()` writes `session.json` at the top of every pass,
and right after the reply, **before** any tool runs.

If the harness dies mid-command, the file already says what was asked.

Written atomically

Write to a temporary file.

Then `os.replace` it.

Either the old file or the new one. **Never half of one.**

Why atomic matters

A crash during a save must not corrupt the save.

That would lose what you were protecting.

unfinished()

At startup, look for a saved session.

If there is one, **ask** whether to pick it up.

Resumed, not restarted

The saved `working_memory` goes back in.

The loop continues from there.

A stale file

If the saved record ends in an assistant message with **no tool request**, the run was finished.

`unfinished()` **deletes** it.

The hard case

The harness died **while a tool was running**.

Did the command run?

We don't know

It may have finished.

It may never have started.

It may have run halfway.

So we say exactly that

The interrupted request gets a `tool_result` with `is_error`:

"may or may not have run"

Honest, not clever

The model reads it and can **check**.

Rerun? List the files? Its choice.

Every tool_use needs a result

An API rule.

A request with no result is malformed.

So resume always supplies one.

Run it

Start in a scratch folder: the box mounts it, and `session.json` goes there.

```
mkdir /tmp/demo && cd /tmp/demo && echo notes > notes.txt
uv run --project /path/to/building-agents \
  /path/to/building-agents/production/08-resilience/quark.py \
  "How many bytes is notes.txt? Answer in one line."
```

Testing failures on purpose

We can't wait for the API to fail.

So the README uses a **stub API**.

flaky.py

A small server that fakes failures.

Otherwise it forwards to the real API.

Pointing quark at it

```
ANTHROPIC_BASE_URL=http://127.0.0.1:PORT
```

The SDK reads it.

No change to `quark.py` .

Stub modes

- `all` : every request fails with 529
- a model name: only that model fails
- `first:N` : the first N requests fail

Demo: the preferred model is down

The stand-in fails every request to `claude-sonnet-5-5`.

The run looks normal.

The trace shows Sonnet failing, and the backup answering.

Demo: the main model is down

Sonnet fails with 529.

quark moves on to **Opus**.

The task completes.

Demo: everything is down

Both models fail.

quark stops with `Down` .

The session is saved.

Demo: kill it

```
kill -9 mid-run.
```

Run quark again.

It resumes.

Going further

`quark.py` is one way to do it.

`resilience.py` is a richer one.

resilience.py

Built on Lesson 3's `control_flow.py` .

112 lines.

Your own retry loop

```
max_retries=0 .
```

The SDK doesn't retry.

The harness does, so you control it.

Backoff

Wait longer after each failure.

Each retry waits more than the last.

Jitter

Add a random amount.

Many clients retrying together would all hit the server at once.

retry-after

A **429** can say how long to wait.

The harness obeys it.

Benching a model

A model that failed is **benched for 60 seconds**.

Later calls skip it and go to the backup.

Why bench

Don't retry a model that just said it was down.

Don't waste time on it.

Come back after a minute.

Tool timeouts

`run()` runs each command with a **timeout**.

Kill the whole group

```
killpg .
```

A command may start children.

Killing just the parent leaves them running.

Output cap

```
MAX_OUT .
```

A command that prints forever can't fill memory.

Checkpoints

After each step, write `checkpoint.json`.

Resume

```
uv run production/08-resilience/resilience.py resume
```

Reads the checkpoint.

Continues.

Demos in the README

- first two requests fail with 429 and a retry-after
- Sonnet down: benched for 60 seconds, Opus answers
- everything down, then `resume`
- a slow tool, timed out and capped
- `kill -9`, then `resume`

What resilience can be

- **how many** retries, and how they wait
- **which** backup models
- **how long** before giving up on a tool
- **what survives** a crash

What to take away

Rule: resilience retries, falls back, reports tool failures, and saves progress, so a failure costs a retry, not the run.

Notice what resilience never does

It never decides **when** to call.

It never decides **what goes in**.

It never edits the working memory it saves.

Its one note to the model, *interrupted, may or may not have run*, is a tool result.

What's missing

The agent survives.

But it's slow, and it costs a lot.

Every step resends everything.

Next: Performance

Lesson 9 makes it faster and cheaper.